

OpenStack Build Reference

OpenStack Build Reference

*House of the Cloud, Hexworth Prime. The companion to the **OpenStack CLI Field Guide** — that one teaches you to read a cloud, this one teaches you to build in it.*

Every command on this page is taken from the graded live labs in this course and verified against this sandbox.

First: which cloud are you in?

This is the single most common source of confusion, and it is not your fault. There are **two** different clouds behind the terminal, and they behave differently.

Mode	What you can do	Where you meet it
Read-only demo	List, show, describe. Every write returns 403 Forbidden.	The Live Cloud Terminal on the OpenStack hub, for exploring.
Personal cloud	Full create, modify, delete — inside your own project only.	The graded live labs. You get your own project when you launch one.

If a `create` command returns **403 Forbidden**, you are almost certainly in the read-only demo, not doing anything wrong. Launch the lab to get a personal cloud. Everything in this handout requires a personal cloud.

The constants of your sandbox

You do not have to discover these every time. In your personal cloud they are fixed.

Thing	Value	Why it matters
Flavor	<code>m1.nano</code>	1 vCPU, 192 MB RAM, 1 GB disk. The <i>only</i> flavor that fits your quota.
Image	<code>cirros-0.6.3-x86_64-disk</code>	A tiny Linux built for testing. Confirm with <code>openstack image list</code> .
Tenant network	<code>shared</code>	Already exists, DHCP on. Use it when a lab just needs a server on <i>some</i> network.
External network	<code>public</code>	The way out. Router gateways and floating IPs come from here.
Decoy network	<code>lab-net</code>	Exists to make <code>--network</code> mandatory. Do not boot onto it — see below.
Quota	1 instance, 1 core, 192 MB	One server at a time. See below.

Why `m1.nano` and nothing else. Your quota caps RAM at 192 MB, and `m1.nano` is exactly 192 MB. Every other flavor on the cloud is larger, so every other flavor is rejected at create time. These are the ones you will see listed, and the reason each fails:

Flavor	RAM	Fits the 192 MB quota?
<code>m1.nano</code>	192 MB	Yes — use this
<code>m1.micro</code>	256 MB	No
<code>cirros256</code>	256 MB	No
<code>m1.tiny</code>	512 MB	No
<code>m1.small</code> and larger	2 GB+	No

There is also exactly one bootable image, `cirros-0.6.3-x86_64-disk`, so if `image list` shows you something else, ask your instructor before using it.

The one-instance rule shapes everything. Your quota allows exactly one running server. If a lab asks you to launch a second one, you must delete the first. If a previous attempt left a server behind, your next `create` fails with a quota error that looks like a bug but is not. Always start with:

```
openstack server list
```

Launch a server

Three questions every server must answer: what size, what operating system, and which network.

```
# 1. What can I boot? (note the exact image name)
openstack image list
```

```
# 2. What networks exist?
openstack network list
```

```
# 3. Launch it
openstack server create \
  --flavor m1.nano \
  --image cirros-0.6.3-x86_64-disk \
  --network shared \
  my-vm
```

```
# 4. Watch it come up -- BUILD, then ACTIVE
openstack server show my-vm -f value -c status
```

A server does not exist the instant the command returns. It goes `BUILD` → `ACTIVE`, and it can land in `ERROR`. When it does, the reason is in the `fault` field:

```
openstack server show my-vm -f value -c fault
openstack server show my-vm -f value -c addresses    # what IP did it get?
```

Why `--network` is not optional here

Leave `--network` off and the cloud refuses:

Multiple possible networks found, use a Network ID to be more specific. (HTTP 409)

That is deliberate. When a cloud has exactly one shared network, Nova quietly attaches it for you and `--network` appears optional — a habit that breaks the moment you touch a real multi-network cloud. This sandbox carries a second shared network, `lab-net`, specifically so Nova has to refuse to guess and you have to say what you mean.

Do not boot onto `lab-net`. Its subnet is created with `--no-dhcp`, so an instance attached to it comes up `ACTIVE` with **no IP address at all** and nothing obviously wrong. It exists to force the choice, not to host workloads. Use `shared`, or a network you create yourself.

Networks, subnets, and routers

These three are a dependency chain, and building them out of order is the most common way to get stuck. Read the order as a rule, not a suggestion:

network (the L2 segment) → **subnet** (the IP range inside it) → **router** (the way off it)

1. Create the network

```
openstack network create lab5-net
```

A network on its own carries no addresses. Nothing can get an IP yet.

2. Give it a subnet

```
openstack subnet create lab5-subnet \  
  --network lab5-net \  
  --subnet-range 10.55.0.0/24
```

The subnet is where DHCP and addressing actually live. `--network` says which network it belongs to; `--subnet-range` is the CIDR block it hands out.

3. Create a router and connect both sides

```
# the router itself  
openstack router create lab5-router  
  
# the INSIDE: attach it to your subnet  
openstack router add subnet lab5-router lab5-subnet  
  
# the OUTSIDE: point its gateway at the external network  
openstack router set lab5-router --external-gateway public  
  
# confirm the gateway actually attached  
openstack router show lab5-router -f value -c external_gateway_info
```

A router has **two** distinct connections and they are configured by **two different commands**. `add subnet` is the inside; `set --external-gateway` is the outside.

Half-built routers are silent. A router with only one side connected still exists and still looks fine in `router list`. Nothing warns you that traffic will never route. The `show` line above is the check that catches it — run it every time.

Useful reads

```
openstack network list          # everything you can see
openstack network list --share  # the ones the cloud lends everyone
openstack network list --external # the ones floating IPs can come from
openstack subnet list
openstack router list
```

Keypairs and floating IPs

A keypair is how you log into a server; a floating IP is how you reach it from outside.

```
# SSH keypair -- do this BEFORE launching, then pass --key-name
openstack keypair create mykey > mykey.pem
chmod 400 mykey.pem

openstack server create --flavor m1.nano --image cirros-0.6.3-x86_64-disk \
  --network shared --key-name mykey my-vm
```

The `provider` trap — read this before you lose an hour

Much OpenStack documentation, including the official Install Guide, allocates floating IPs from a network called `provider` and builds tenant networks called `selfservice`. **Those names do not exist in this cloud.** Here the external network is `public` and the ready-made tenant network is `shared`.

What goes wrong: `openstack floating ip create provider` fails, so it is natural to think “the network is missing” and run `openstack network create provider` to fix it. That command *succeeds* — and leaves you worse off. A network you create is a tenant network: it has no subnet, and it can never be external, because only an administrator can mark a network external. You now own a dead-end network with the right name and none of the properties.

```
openstack network delete provider  # the dead end
openstack network list --external  # the only column that matters
openstack floating ip create public
```

Network	External	Subnet	Use it for
public	yes	172.24.4.0/24 (v4) and an IPv6 subnet	Floating IPs and router gateways. This is the one.
shared	no	192.168.233.0/24, DHCP on	Attaching instances. The normal choice.
lab-net	no	10.99.0.0/24, DHCP off	Nothing, deliberately — see below.
provider <i>(only if you created one)</i>	no	none	Nothing. Delete it.

A floating IP allocated from `public` comes out of the IPv4 pool 172.24.4.2–172.24.4.254, so expect a 172.24.4.x address. The network also carries an IPv6 subnet, which is why `network list` reports two.

The full path from nothing to a reachable server

This exact sequence is verified working in this sandbox:

```
openstack network create my-net
openstack subnet create my-subnet --network my-net --subnet-range 10.55.0.0/24
openstack router create my-router
openstack router add subnet my-router my-subnet          # inside
openstack router set my-router --external-gateway public # outside
openstack server create --flavor m1.nano \
  --image cirros-0.6.3-x86_64-disk --network my-net my-vm
openstack floating ip create public                      # yields a 172.24.4.x address
openstack server add floating ip my-vm <THE-IP>
```

If you attached the IP and still cannot reach the server, check the router gateway before you suspect the IP.

Security groups

A security group is a firewall attached to a server. New groups start closed — they permit nothing inbound until you add a rule.

```
openstack security group create web-sg --description "web tier"
# allow SSH in, but only from inside 10.0.0.0/8
```

```
openstack security group rule create web-sg \  
  --protocol tcp \  
  --dst-port 22:22 \  
  --remote-ip 10.0.0.0/8 \  
  --ingress  
  
# check what you actually created -- ports AND source both matter  
openstack security group rule list web-sg  
  
# attach it to a running server, then confirm it stuck  
openstack server add security group my-vm web-sg  
openstack server show my-vm -f value -c security_groups
```

--dst-port 22:22 is a *range* written start:end. A single port is written as the same number twice. --remote-ip is who is allowed to connect; leaving it off means everyone, which is rarely what you want and is usually what a grader is checking for.

Compare yours against the default group to see what a permissive one looks like:

```
openstack security group rule list default
```

Volumes

A volume is a disk that outlives the server it is attached to. That independence is the entire point, and it is also what makes the attach/detach order matter.

```
openstack volume create --size 1 lab-vol  
openstack server add volume my-vm lab-vol  
  
# available = free, in-use = attached  
openstack volume show lab-vol -f value -c status  
  
# detach BEFORE deleting the volume  
openstack server remove volume my-vm lab-vol  
openstack volume delete lab-vol
```

A volume cannot be attached to two servers at once, and an `in-use` volume cannot be deleted. If `volume delete` refuses, run `openstack volume show lab-vol` — the status field will say `in-use` and the attachments field will tell you which server is holding it.

Teardown, in the only order that works

OpenStack refuses to delete anything that something else still depends on. It does not cascade for you. Tear down in the reverse of the order you built.

```
# 1. servers first (they hold ports, volumes, and IPs)
openstack server delete my-vm

# 2. detach volumes, then delete them
openstack server remove volume my-vm lab-vol
openstack volume delete lab-vol

# 3. unplug the router -- BOTH sides
openstack router unset --external-gateway lab5-router
openstack router remove subnet lab5-router lab5-subnet
openstack router delete lab5-router

# 4. now the network will go
openstack network delete lab5-net
```

The trap that costs the most time. A router with an interface still attached refuses to delete. A network with a router port on it refuses to delete. If you skip `router remove subnet`, the teardown fails — and your next attempt creates a *second* `lab5-net`. From then on, every command that names `lab5-net` is ambiguous and errors out, and the lab becomes unrunnable until you delete by ID instead of name. Check with `openstack network list` before you rebuild.

Error decoder for writes

What you see

403 Forbidden

Quota exceeded on server create

No valid host was found

Multiple ... found / ambiguous name

What it means

Your role cannot write here.

You already have a server.

Nothing can fit the flavor you asked for.

Two objects share that name.

What to do

You are in the read-only demo.
Launch the lab for a personal cloud.
`openstack server list`, delete the leftover,
retry.
Use `m1.nano`. Larger flavors exceed
your 192 MB.

What you see	What it means	What to do
Router or network <code>in use</code>	Something is still attached.	List with IDs and delete by ID, not name. Follow the teardown order above, both router sides.
Volume delete refused	Volume is <code>in-use</code> .	<code>server remove volume</code> first, wait for available.
Floating IP create fails	That network is not external.	<code>openstack network list --external</code> , then use <code>public</code> .
Multiple possible networks found (409)	You omitted <code>--network</code> .	Say which one: <code>--network shared</code> . This is deliberate — see above.
Server is <code>ACTIVE</code> but has no IP	You booted onto <code>lab-net</code> , which has DHCP disabled.	Delete it and relaunch with <code>--network shared</code> .
Server stuck in <code>ERROR</code>	Scheduling or boot failed.	<code>openstack server show <name> -f value -c fault</code> for the reason.

If a command here disagrees with a lab, the lab is authoritative — tell your instructor so this handout gets corrected.